Short refresher



Promises

40 minutes

Async | Week 4 | Mandatory | Not Completed



Await / Async

11 minutes

Async | Week 4 | Mandatory | Not Completed



Fetch

10 minutes

AJAX | Week 4 | Mandatory | Not Completed

Promises

Promises

- Asynchronous JavaScript
  - Some operations produce their result later
    - Network requests
  - JavaScript represents a future result using a `Promise`
    - **Pending**
      - The result is not available yet
    - **Fulfilled**
      - The operation succeeded
      - The promise contains a value
    - **Rejected**
      - The operation failed
      - The promise contains a reason or error
    - **Settled**
      - General term meaning either fulfilled or rejected

Promises

- Working with a single promise
  - `.then()` handles fulfilment
    - Every `.then()` call returns a new promise
- Rejections and promise chains
  - A rejection travels down the chain
    - Success handlers are skipped
    - The next rejection handler receives the error
  - Errors and exceptions cause promise rejections
  - `.catch()` handles a rejection
    - It can pass the error to a later `.catch()` by rethrowing
    - More operations can occur after a `.catch()`
  - `.finally()` runs after either outcome

Promises

- Working with multiple promises
  - `Promise.all`
    - Returns a promise that resolves only if all supplied promises resolve
  - `Promise.allSettled`
    - Returns a promise after all supplied promises have settled
  - `Promise.race`
    - Races the supplied promises
  - `Promise.any`
    - Returns a promise that resolves if at least one supplied promise resolves

Fetch

Fetch

- Fetching data
  - `fetch()` returns a promise
    - It fulfils when an HTTP response is received
    - It rejects when there is a network error
  - An HTTP error is still an HTTP response
    - A status other than `200` does not automatically reject the fetch promise
    - The promise fulfils with a `Response`

Async / Await

Async / Await

- Async and await
  - `async` and `await` are built on promises
    - They do not replace the promise model
    - They provide another syntax for using it
  - An `async` function always returns a promise
  - `await` consumes a value or promise
    - Fulfilled promise
      - Produces the fulfilled value
    - Rejected promise
      - Throws the rejection reason at the `await` expression
    - Non-promise value
      - Treated like an already-fulfilled promise
  - The runtime suspends the async function at `await`
    - It returns a promise to the caller
    - It later resumes the function from that point

Async / Await

- Error handling with async and await
  - Rejections at an `await` expression are thrown as exceptions
  - Errors and awaited rejections can be handled with `try/catch`
  - `try/catch` only catches the rejection if the promise is awaited inside the `try`

Async / Await

- Execution ordering
  - Promise handlers run asynchronously
    - The current function context completes first
  - `await` suspends the current async function
    - The async function returns a promise to its caller
    - The suspended function later resumes from the point where `await` was used

Exercise: async-error-handling



Do some work combining javscript timeouts with promises

20 minutes

Async Week 5 Compulsory

Exercise: async-promises-timeouts



Learn how to handle promise errors with .catch and .finally

15 minutes

Async Week 5 Compulsory

Exercise: async-promises-all



Learn how to use promise.all to handle multiple promises concurrently

25 minutes

AJAX | Week 5 | Compulsory

Exercise: async-nested-promises



Learn about how to use the result of promises to make more promises

25 minutes

Async Week 5 Compulsory